

Support: moteur Portescap référencé **35GLT2R82-426P.1**

mass: 350g		35GLT2R82 ☉☉ • 1				
Winding Type	☉☉	426P	326P	234E	426SP	426E
Measured Values						
Measuring voltage	V	24	24	48	48	90
No-load speed	rpm	5800	5800	7500	6200	5500
Stall torque	mNm	1421	1053	1300	1409	1487
Average No-load current	mA	120	120	70	60	40
Typical starting voltage	V					
Max. Recommended Values						
Max. continuous current	A	4.10	3.60	2.20	2.10	1.05
Max. continuous torque	mNm	155.6	137.5	129.9	151.0	158.6
Max. angular acceleration	10 ³ rad/s ²	148	140	160	139	185
Intrinsic Parameters						
Back-EMF constant	V/1000 rpm	4.09	4.09	6.37	7.74	16.40
Torque constant	mNm/A	39.1	39.1	61.0	74.0	157.0
Terminal resistance	Ohms	0.66	0.9	2.25	2.52	9.5
Motor regulation R/k ²	10 ³ /Nms	0.43	0.59	0.61	0.46	0.39
Rotor inductance	mH	0.10	0.15	0.25	0.40	1.70
Rotor inertia	kgm ² 10 ⁻⁷	83.00	75.00	65.00	56.00	70.00
Mechanical time constant	ms	3.6	4.4	4.0	4.0	2.7

Activité : Avec Python sur Spyder, **éditez** et **testez** une fonction qui prend comme arguments la vitesse et le couple utile. Cette fonction renvoie la tension à appliquer au moteur.

```
import matplotlib.pyplot as plt
import numpy as np

R=0.66      # Résistance induit (rotor)
k=39.1e-3   # Constante de couple ki ou de vitesse ke
Cp=4.7e-3   # Couple de pertes considéré constant
Um=24       # Tension nominale moteur

Vmax= 5800      # Vitesse maximale à vide en tr/min , couple nul
Cmax = 1.421    # Couple de démarrage en Nm à vitesse nulle

'Cette fonction prend comme arguments la vitesse et le couple et renvoie la tension'
def tension(vitesse:float,couple:float):
    return R*couple/k + k*vitesse      # Modèle RE avec vitesse en rad/s

'Test de la fonction sur deux cas connus Couple max et vitesse max'
print(tension(Vmax*np.pi/30,0))      # Renvoie +24V
print(tension(0,Cmax))                # Renvoie +24V
```

Activité : Avec Python sur Spyder, **éditez** un premier script qui, pour une tension Um donnée, calcule la vitesse en fonction du couple variant de 0 à Cmax. Le couple maximal est fonction de la tension d'alimentation.

Tracez la vitesse en fonction du couple avec matplotlib en renseignant les axes et unités.

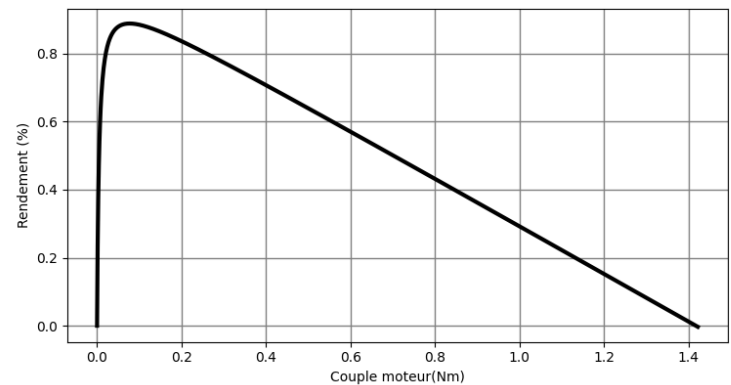
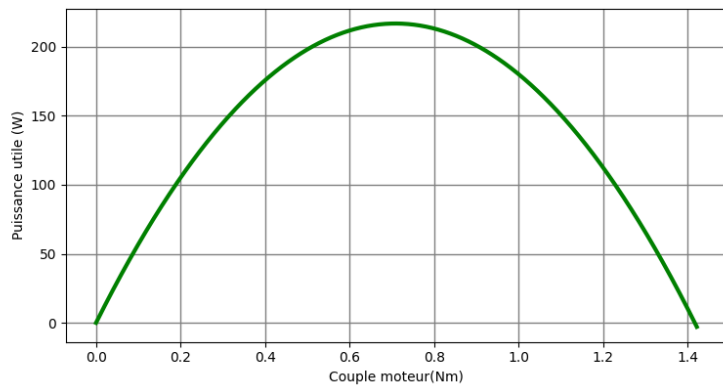
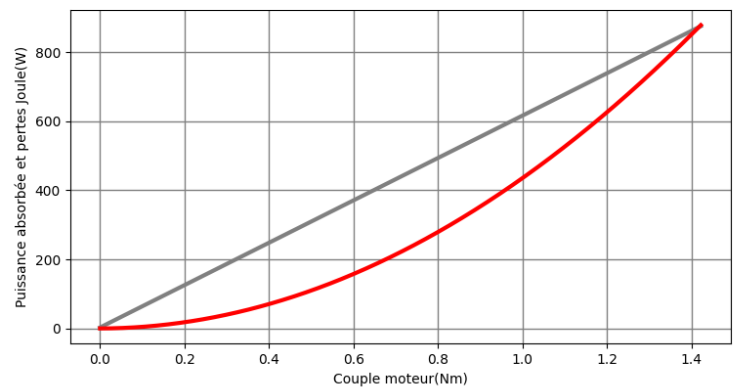
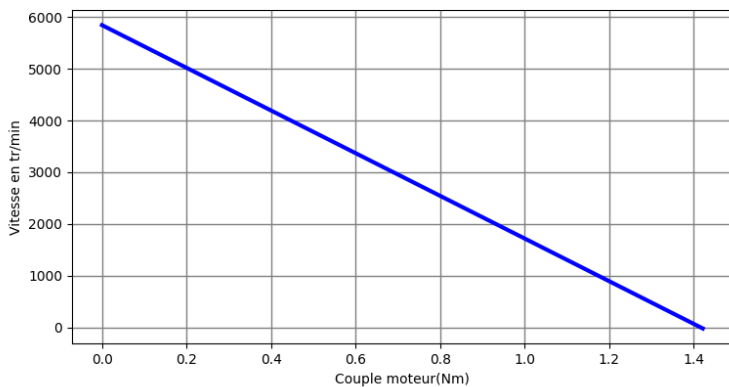
Tracez les pertes Joule en fonction du couple puis la puissance absorbée, la puissance utile et enfin le rendement.

```
Cmax=k*Um/R      # Calcul couple de démarrage
Couple_resistant=np.linspace(0,Cmax,1000)      # 1000 valeurs de couple
Couple_moteur=Couple_resistant+Cp      # Pas de frottements fluides
Couple_utile=Couple_resistant      # Régime permanent
vitesse= 30*(Um - R*Couple_moteur/k)/(k*np.pi)      # Relation vitesse = f(couple)

plt.subplot(2,2,1)
plt.plot(Couple_utile,vitesse,color='b',linewidth='3') #Vitesse en fonction du couple
plt.grid(color='grey', linestyle='-', linewidth=1)
plt.xlabel("Couple moteur(Nm)")
plt.ylabel("Vitesse en tr/min")
```

```
plt.subplot(2,2,2)
puis_abs=Um*Couple_moteur/k          # Pa = U.I
puis_joule=R*(Couple_moteur/k)**2    # Pj= RI2
plt.plot(Couple_utile,puis_abs,color='grey',linewidth='3')
plt.plot(Couple_utile,puis_joule,color='red',linewidth='3')
plt.grid(color='grey', linestyle='-', linewidth=1)
plt.xlabel("Couple moteur(Nm)")
plt.ylabel("Puissance absorbée et pertes Joule(W)")
```

```
plt.subplot(2,2,3)
puis_utile= Couple_utile*vitesse*np.pi/30    # Pu=CΩ
plt.plot(Couple_utile,puis_utile,color='green',linewidth='3')
plt.grid(color='grey', linestyle='-', linewidth=1)
plt.xlabel("Couple moteur(Nm)")
plt.ylabel("Puissance utile (W)")
```



Constats :

- le courant est à l'image de la puissance absorbée pour $U_m = +24V$
- les pertes Joule augmentent avec le carré du courant
- la puissance utile est nulle à vide et rotor bloqué
- le point de fonctionnement est soit à la puissance maximale soit au rendement maximal
- le fonctionnement nominal est donné pour un couple de 0,155Nm dans la zone de rendement maximal

Fichier *MoteurTraitementNumCorrigé.py*